Roll No. 25B0951

Name: Kushagra Gupta

Seat: LA 001-N16

Marks obtained

Data Structures and Algorithms

CS213 (Autumn 2026)

27 August 2026 - Quiz 1

Instructor: Prof. Manoj Prabhakaran

Total: 25 points | *Duration: 55 min*

A. Short Answer Questions

1. Which of the following is/are correct: [2 pts]

   If $f(n) = O(n)$ then $f(n) \leq n^2$ for large enough n.

   If $f(n) = O(n)$ then $f(n) \geq \log n$ for large enough n.

   If $f(n) = O(n)$ then $f(n) \leq n$ for large enough n.

   If $f(n) = O(n)$ then $f(n) = O(n^2)$.

2. Given two singly linked lists (as pointers to head and tail for each of them) of n elements each, how much time does it take to produce a list which has all the elements in the two lists combined (duplicates are allowed). Choose the tightest bound below. [1 pt]

   $O(1)$ time

   $O(\log n)$ time

   $O(n)$ time

   $O(n^2)$ time

3. Each time an `std::vector` object runs out of space, a new vector of double the capacity is allocated and data is moved to the new vector. What is the ***total time*** needed to insert n elements into an `std::vector` object? Choose the tightest bound from below. [1 pt]

   $O(1)$ time

   $O(n)$ time

   $O(n \log n)$ time

   $O(n^2)$ time

N16 - LA 001
25B0951
Kushagra Gupta

Marks obtained

4. Something is wrong with the following definition of the big-O notation. Fix it and describe what you changed. [3 pts]

> $f(n) = O(g(n))$ if there is a threshold t such that for each $n > t$, there exists a number c such that $f(n) \leq c\, g(n)$.

Fixed version:

What did you change?

5. Consider the implementation of Deque from the lectures, using a collection of fixed-capacity Deques, and an additional fixed-capacity Deque as a "directory". [2 pts]

Select all the correct statements about this implementation.

- push_back, push_front, pop_back and pop_front operations take worst-case O(1) time.
- Random access takes worst-case O(1) time.
- Inserting and erasing at any position in the deque takes amortised O(1) time.
- The fixed-capacity deques holding the entries in the deque are periodically reallocated to double their capacity.

N16 - LA 001
25B0951
Kushagra Gupta

Marks obtained

B. Build a queue from two stacks

Consider the class `QueueUsingStacks` below, in which a queue is implemented using two `std::stack` objects, `in_stack` and `out_stack`. For simplicity, we will only implement `enqueue()` and `dequeue()` functions in `QueueUsingStacks` (omitting the `front()` function). **The problem has two parts.** [8 pts]

1. The `enqueue()` function is already implemented for you: it simply pushes incoming items into the `in_stack`. You need to complete the implementation of the `dequeue()` function by filling in the designated blank space. (Do not modify the class in any other way.) As the provided code shows, dequeuing involves popping `out_stack`. The code you need to fill in could take O(n) time in the worst-case, but should be O(1) when amortised over all the enqueue and dequeue operations.

   For your reference, you can use the following functions (shown here with simplified signatures) from `std::stack<int>` (pop and top shouldn't be called on an empty stack):

   ```
   bool empty();  void push(int val); void pop(); int top();
   ```

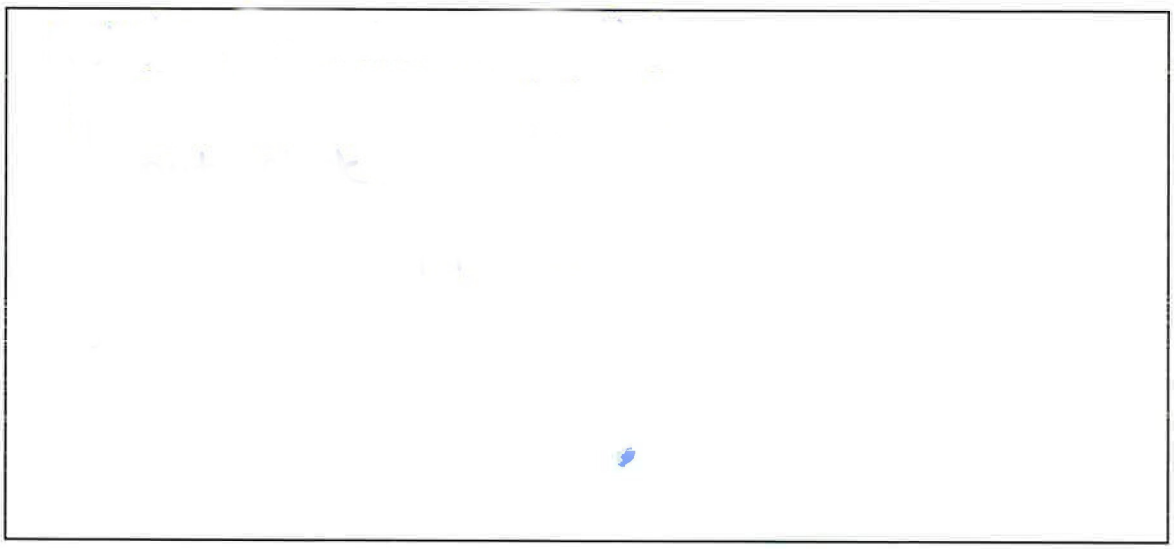
```
#include <stack>
class QueueUsingStacks {
    std::stack<int> in_stack;
    std::stack<int> out_stack;

public:
    //------- The queue is empty if both the stacks are empty ------
    bool empty() const {
        return in_stack.empty() && out_stack.empty();
    }
    //-------  Newly enqueued elements go into the in_stack -------
    void enqueue(int value) {
        in_stack.push(value);
    }
```

N16 - LA 001
25B0951
Kushagra Gupta

Marks obtained

```
//------- Remove the element from the front of the queue -------
// This is done by removing the element from the top of out_stack.
// Handle the case when out_stack is empty but in_stack is not,
// such that the amortized cost of dequeueing is O(1)
bool dequeue(int& value) {
    if (empty())
        return false;
    if (out_stack.empty()) { // Fill in the code below
```



```
    }
    value = out_stack.top(); // This is the dequeued value
    out_stack.pop();
    return true;
}
};
```

N16 - LA 001
25B0951
Kushagra Gupta

Marks obtained

2. Give an argument to prove that the *amortised* cost of your `dequeue()` function is O(1).

C. Build two stacks from one deque

In this problem, we implement two stacks using a single deque. The stacks we are implementing have unbounded capacity; the deque we are using also has unbounded capacity. For simplicity, we implement only two functions, push and pop for the stacks, with the latter function delivering the element that is popped. Both of these functions take an extra argument to indicate whether the operation is on the first stack or the second stack. [8 pts]

N16 - LA 001
25B0951
Kushagra Gupta

Marks obtained

For your reference, you can use the following functions (shown here with simplified signatures) from `std::deque<char>`:

```
bool empty(); unsigned int size(); char front(); char back();
void push_front(char val); void push_back(char val);
void pop_front(); void pop_back();
```

```
#include <deque>
class TwoStacksUsingDeque {
    std::deque<char> dq; // Elements of Stack 1 and Stack 2 live here

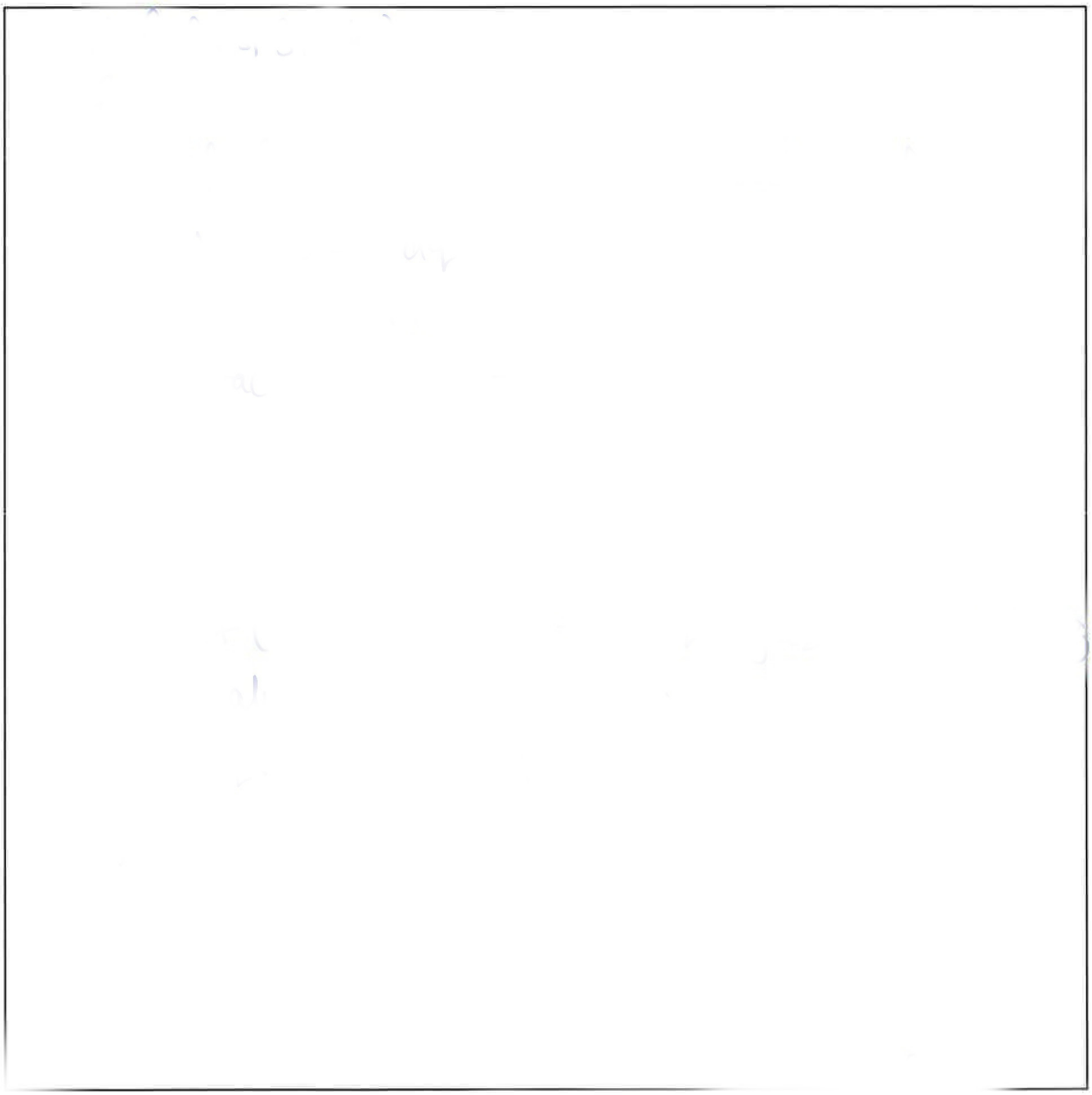
    // Stack 1's element count (Stack 2's count is dq.size() - stack1_size)
    unsigned int stack1_size = 0;
public:
    //-------  To push an item into Stack 1 or Stack 2. -------
    // If firstStack is true, push to Stack 1. Else into Stack 2.
    void push(char val, bool firstStack) { // Fill in below
```

N16 - LA 001
25B0951
Kushagra Gupta

Marks obtained



```
//-------  Pops the top element from the specified stack -------
// Popped element is copied to the argument value.
// If the specified stack is empty, returns false.
bool pop(char& value, bool firstStack) { // Fill in below
```



```
};
```